

Virtus Polímeros API

Documentación técnica y guía de traspaso para desarrolladores

Proyecto	Virtus Polímeros API
Stack	CodeIgniter 4 · PHP · MySQL · JWT · OpenAI
Repositorio	github.com/manuelrojasm/virtus-polimeros-api
Autor	Manuel Alberto Rojas Martínez
Supervisor	Jaime Alberto Paez (Docente – ETITC)
Versión del doc.	1.0 – abril 2026
Audiencia	Desarrolladores que reciben el proyecto

Plataforma web para la capacitación y concientización en la contaminación por polímeros y el uso de biopolímeros.

Contenido

1. Introducción y objetivos
2. Stack tecnológico
3. Arquitectura y estructura del proyecto
4. Configuración del entorno y despliegue
5. Base de datos y modelos
6. Autenticación y autorización (JWT)
7. Endpoints de la API
8. Servicios de negocio
9. Integración con OpenAI
10. Almacenamiento de archivos
11. Convenciones, pendientes y recomendaciones
12. Contacto

1. Introducción y objetivos

Virtus Polímeros API es el backend de una plataforma web cuyo propósito es educar y concientizar a la comunidad sobre el impacto ambiental de los polímeros sintéticos y promover el uso de biopolímeros como alternativa sostenible. La plataforma centraliza el material didáctico (cursos, secciones con PDFs, preguntas y evidencias) y registra el progreso de los estudiantes que completan las capacitaciones.

Este documento está dirigido a un desarrollador que recibe el proyecto en frío y necesita entender cómo está construido, cómo levantarlo, dónde tocar para extenderlo y qué decisiones técnicas ya se tomaron. No es un manual de usuario final ni un manual de operaciones; es una guía de lectura del código.

Qué encontrarás

- Un mapa del stack y de cómo se organiza el código (convenciones de CodeIgniter 4).
- Esquema relacional de la base de datos **mydb** y cómo los modelos se conectan a ella.
- Listado completo de endpoints con su autorización esperada y ejemplos de uso.
- Flujos críticos: autenticación JWT y gestión de cursos (con ingesta de PDFs y generación de preguntas con OpenAI).
- Guía de configuración (.env), despliegue y recomendaciones de mantenimiento.

2. Stack tecnológico

El backend está construido con **CodeIgniter 4** (PHP ≥ 7.4 , compatible con PHP 8.x). Se expone como una API REST que consume una base de datos **MySQL** (driver MySQLi) y se integra con servicios externos de OpenAI y SMTP. Las dependencias se manejan con Composer; las principales son:

Dependencia	Versión	Uso en el proyecto
codeigniter4/framework	^4.0	Framework principal (MVC + RESTful)
firebase/php-jwt	^6.11	Emisión y validación de tokens JWT
openai-php/client	^0.19	Cliente oficial de OpenAI (Files + Responses API)
guzzlehttp/guzzle	^7.10	HTTP client (usado por OpenAI)
php-http/guzzle7-adapter	^1.1	Adaptador Guzzle para PSR-18
smalot/pdfparser	^2.12	Utilidades para lectura de PDFs
zircote/swagger-php	^5.1	Generación del spec OpenAPI desde atributos
phpunit/phpunit	^9.1 (dev)	Pruebas unitarias
fakerphp/faker	^1.9 (dev)	Datos de prueba para seeders

Documentación OpenAPI: los controladores usan atributos PHP `#[OA\...]` para anotar cada endpoint. El spec se compila en tiempo de build a **swagger.json** mediante *zircote/swagger-php*.

3. Arquitectura y estructura del proyecto

El proyecto sigue el layout estándar de CodeIgniter 4 con una capa adicional de **Services** para aislar la lógica de negocio que no pertenece ni a los controladores ni a los modelos. Las solicitudes HTTP entran por *public/index.php*, pasan por los filtros globales (CORS) y por filtros de ruta (*auth*, *admin*) antes de llegar al controller correspondiente, que delega a un service y éste a los modelos de Eloquent-style de CI4.

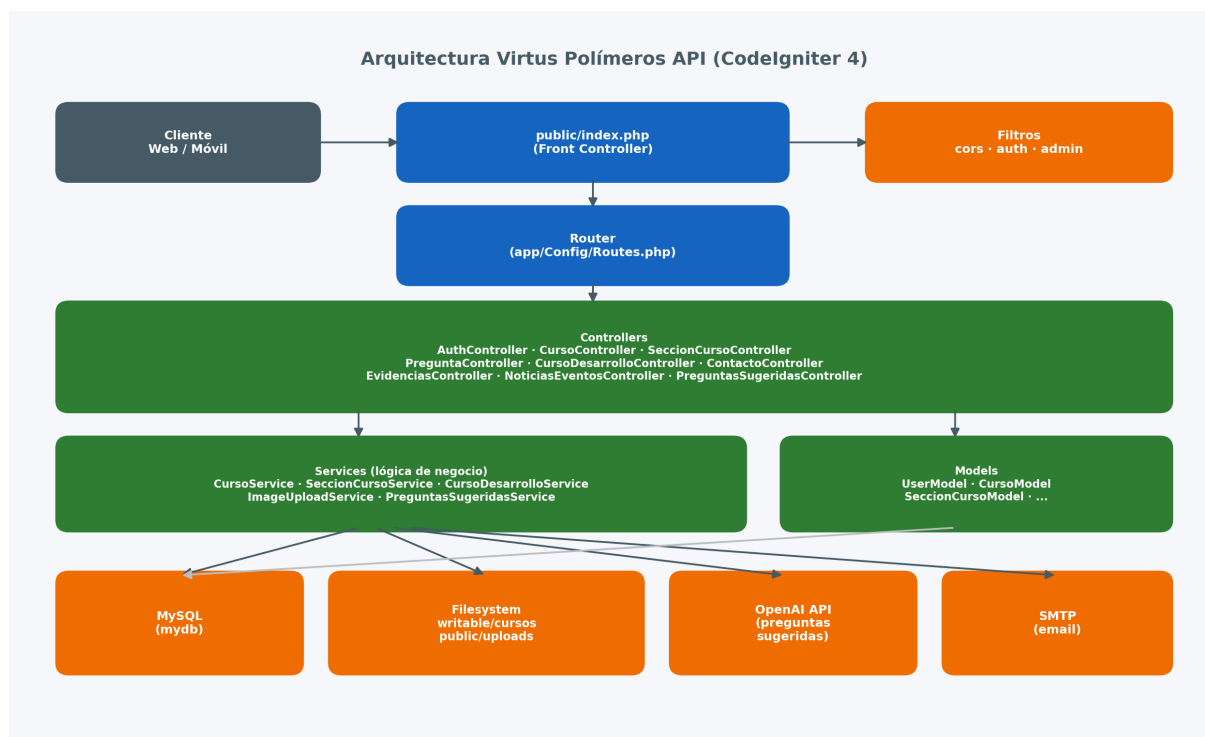


Fig. 1 · Arquitectura en capas del backend.

Layout de directorios

```

virtus-polimeros-api/
├── app/
│   ├── Config/ # Configuración (Routes, Filters, Database, Services...)
│   ├── Controllers/ # 11 controladores REST
│   ├── Models/ # 9 modelos (uno por tabla principal)
│   ├── Services/ # 5 services de negocio
│   ├── Filters/ # AuthFilter, AdminFilter, Cors
│   ├── Database/ # Migrations y Seeds
│   ├── Views/
│   └── public/
│       ├── index.php # Front controller
│       ├── uploads/ # Imágenes públicas (portadas, fotos de perfil)
│       ├── writable/
│       │   ├── cursos/{idCurso}/ # PDFs privados de secciones
│       │   ├── cache/ logs/ session/
│       │   └── docs/ # Documentación (incluye este PDF)
│       ├── scripts/ # Utilidades
│       ├── tests/
│       ├── .env # Variables de entorno (no versionado)
│       ├── composer.json
│       ├── scrit BD virtud.sql # DDL de referencia de la BD
│       └── swagger.json
  
```

Convenciones clave

- **Namespaces:** `App\Controllers`, `App\Models`, `App\Services`, `App\Filters`. Cargados vía PSR-4 desde `composer.json`.
- **Nombres de columnas:** *PascalCase* en español (*PrimerNombre*, *FechaCreacion*, *Estado*). Todas las tablas exponen un campo *Estado* (1 = activo / 0 = inactivo) para soft-delete.
- **Respuestas HTTP:** los controllers heredan de `ResourceController` y retornan JSON con la forma `{ success: bool, message: str, data?: ... }`.
- **Servicios:** instanciables con `\Config\Services::curso()`, `::seccionCurso()`, `::imageUpload()`, etc. (registrados en `app/Config/Services.php`).

- **CORS:** habilitado globalmente por el filtro *cors* y reforzado por cabeceras manuales en algunos controllers.

4. Configuración del entorno y despliegue

Requisitos mínimos

- PHP $\geq$ 7.4 (probado con 8.x). Extensiones: *intl*, *mbstring*, *mysqli*, *curl*, *fileinfo*, *gd*.
- Composer 2.x.
- MySQL 5.7+ o MariaDB 10.3+.
- Servidor web Apache o Nginx apuntando a **public/**.
- Cuenta y clave de OpenAI (solo si se usa */cursos/{id}/preguntas-sugeridas*).
- SMTP saliente si se quiere usar el endpoint de recuperación de contraseña.

Instalación local

```
git clone https://github.com/manuelrojasm/virtus-polimeros-api.git
cd virtus-polimeros-api
composer install
cp env .env # Plantilla ya incluida en el repo
mysql -u root -p < "script BD virtus.sql"
php spark migrate # si usas las migraciones de CI4
php spark serve # http://localhost:8080
```

Variables de entorno mínimas

Variable	Descripción
CI_ENVIRONMENT	development production
app.baseURL	URL pública de la API (ej. https://api.virtuspolimeros.co/)
database.default.hostname	Host MySQL
database.default.database	Nombre de la base (por defecto mydb)
database.default.username / password	Credenciales de BD
database.default.DBDriver	MySQLi
JWT_SECRET	Secreto HMAC para firmar JWT (HS256). Obligatorio.
OPENAI_API_KEY	Clave de OpenAI (solo si se usan preguntas sugeridas)
email.*	Configuración SMTP (usuario, contraseña, host, puerto, SMTPCrypto)

Despliegue en producción

- Apunta el *DocumentRoot* del servidor web a **public/**; nunca expongas *app/*, *writable/* ni el *.env*.
- Asegura permisos de escritura en **writable/** y **public/uploads/** para el usuario del web server.
- Configura *CI_ENVIRONMENT* = *production* para desactivar el debug toolbar y errores detallados.
- Ejecuta *composer install --no-dev --optimize-autoloader* al desplegar.
- Protege el endpoint de recuperación de contraseña con rate limiting a nivel de servidor.
- Programa respaldo regular de la base *mydb* y de la carpeta *writable/cursos/* (contiene los PDFs originales).

5. Base de datos y modelos

La BD se llama **mydb** y utiliza *InnoDB*. El DDL de referencia está en *scrip BD virtus.sql* en la raíz del repo. Existen además migraciones en [app/Database/Migrations/](#) que pueden ampliar o ajustar ese DDL; si partes desde cero, lo más rápido es ejecutar el script SQL y luego correr las migraciones para traer los cambios posteriores (campos como *FotoPerfil* o *idCurso* en *pregunta*).

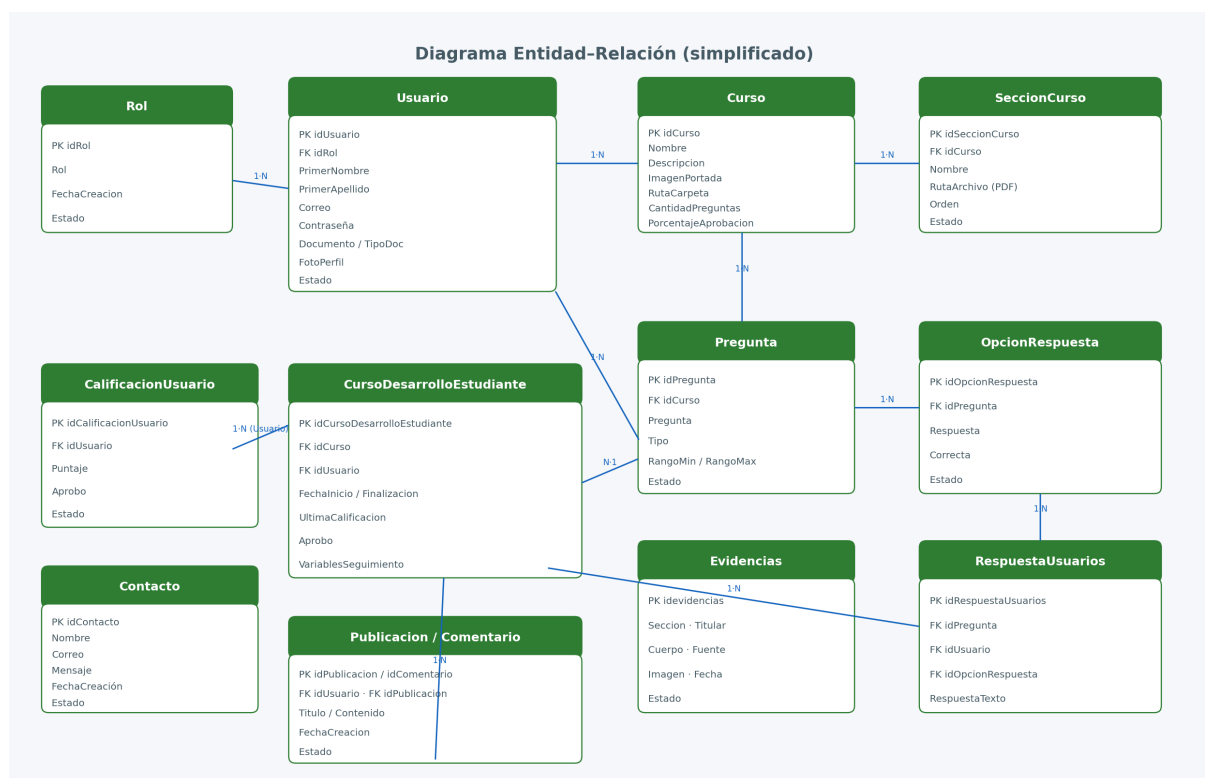


Fig. 2 · Entidades principales y sus relaciones.

Tablas principales

Tabla	Modelo	Propósito
Rol	—	Catálogo de roles. 1 = estudiante, 2 = admin.
Usuario	UserModel	Cuentas de estudiantes y administradores.
Contacto	ContactoModel	Mensajes de contacto desde el formulario público.
Curso	CursoModel	Cursos. Cada uno tiene una carpeta física en <i>writable/cursos/{id}</i> .
SeccionCurso	SeccionCursoModel	Secciones de un curso, cada una con un PDF asociado.
Pregunta	PreguntaModel	Preguntas asociadas a un curso. Tipos: multiple, única, falso_verdadero, autocompletar.
OpcionRespuesta	OpcionRespuestaModel	Opciones disponibles para una pregunta.
RespuestaUsuarios	—	Respuestas efectivas de cada usuario.
CalificacionUsuario	—	Resultados consolidados de un usuario (puntaje, aprobado).
CursoDesarrolloEstudiante	CursoDesarrolloEstudiante Model	Progreso de un estudiante en un curso (inicio, fin, calificación).
Evidencias	EvidenciasModel	Contenido didáctico / divulgación (secciones con titular, cuerpo, fuente, imagen).
noticias_eventos	NoticiasEventosModel	Noticias y eventos institucionales (públicos o restringidos).

Tabla	Modelo	Propósito
Publicacion / Comentario	—	Muro social (sin controlador aún; reservado para futuras features).

Notas importantes

- Todos los *Models* de CI4 apuntan a su tabla con `$table` y a su PK con `$primaryKey`. Los campos permitidos para mass assignment se declaran en `$allowedFields`.
- El modelo **SeccionCursoModel** añade el método `seccionesPorCurso($idCurso)` que ordena por el campo `Orden`.
- Los DDL originales no incluyen `idCurso` en **Pregunta** ni **FotoPerfil** en **Usuario**; estas columnas las agregan las migraciones de CI4 o se definen en los `$allowedFields`. Revisa `app/Database/Migrations/` antes de llevar la BD a producción.
- Soft delete: las filas se marcan con `Estado = 0` en lugar de borrarse físicamente (excepto `Pregunta/OpcionRespuesta`, que sí permiten DELETE para admin).
- Codificación: `utf8` y `utf8_general_ci`; si migras a `utf8mb4` revisa longitudes de índices.

6. Autenticación y autorización (JWT)

La API no mantiene sesión en servidor: emite un **JWT** en el login y espera ese token en cada request protegida dentro del header `Authorization: Bearer <token>`.

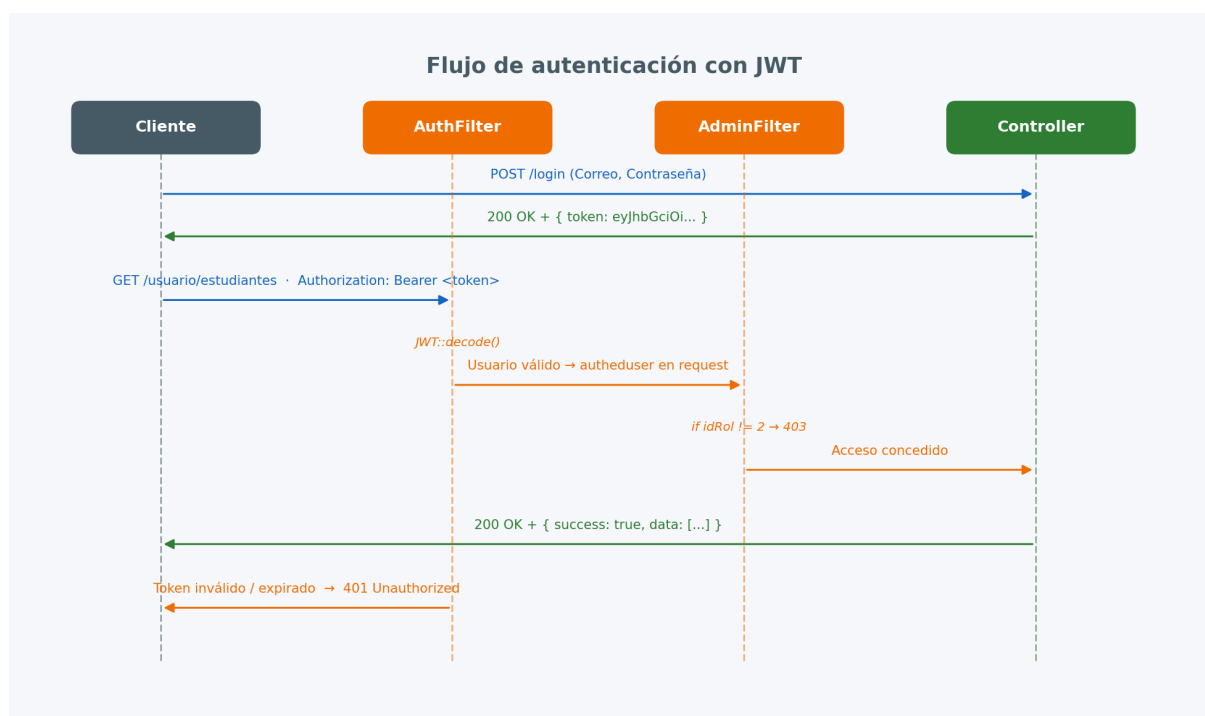


Fig. 3 · Flujo de autenticación y autorización.

Detalles

- Algoritmo: **HS256**. Secreto: variable de entorno `JWT_SECRET`. Caducidad: **1 hora** (3600 s).
- Payload emitido: `{ id, idRol, Nombre, Apellido, iat, exp }`.
- Filtro **AuthFilter** (`app/Filters/AuthFilter.php`): decodifica el token, valida firma y expiración, y deja los claims en `$request->authUser`.
- Filtro **AdminFilter**: verifica `authUser['idRol'] === 2`; debe usarse **después** de `auth` (de ahí el patrón `['filter' => ['auth', 'admin']]`).

- Errores esperados: 401 (token ausente/inválido/expirado), 403 (no es admin).

Reglas de negocio sobre el propietario

Los endpoints que actualizan el perfil, suben foto o cambian contraseña (`/usuario/perfil/{id}`, `/foto`, `/cambiar-clave/{id}`) comparan el `id` de la ruta contra el `id` dentro del JWT. Si no coinciden, responden 403. Esto evita que un usuario modifique datos de otro aun teniendo un token válido.

7. Endpoints de la API

Las rutas viven en `app/Config/Routes.php` y están anotadas con atributos OpenAPI en cada método del controller, lo que permite regenerar la documentación Swagger cuando sea necesario. La siguiente tabla agrupa los endpoints por dominio; la columna **Auth** indica el filtro aplicado: *público*, *auth* (cualquier usuario autenticado) o *admin* (auth + idRol = 2).

7.1 Autenticación y usuarios

Método	Ruta	Auth	Descripción
POST	<code>/api/register</code>	Público	Registra un nuevo usuario (valida correo, documento, edad).
POST	<code>/login</code>	Público	Autentica con correo + contraseña; devuelve JWT.
POST	<code>/recoverPassword</code>	Público	Genera una contraseña aleatoria y la envía por correo.
PUT	<code>/usuario/perfil/{id}</code>	auth	Actualiza nombre/correo/celular. Para borrar foto: FotoPerfil=null.
POST	<code>/usuario/perfil/{id}/foto</code>	auth	Sube foto de perfil (multipart, campo foto; máx 2 MB).
PUT	<code>/usuario/cambiar-clave/{id}</code>	auth	Cambia la contraseña (exige la actual).
GET	<code>/usuario/estudiantes</code>	admin	Lista usuarios con idRol = 1.
PUT	<code>/usuario/estado-usuario-estudiante/{id}</code>	admin	Activa / desactiva un estudiante (toggle).

7.2 Cursos, secciones y progreso

Método	Ruta	Auth	Descripción
GET	<code>/cursos</code>	Público	Lista cursos (filtro opcional por Estado).
GET	<code>/cursos/{id}</code>	Público	Detalle de un curso.
GET	<code>/cursos/activos-con-preguntas</code>	auth	Cursos activos con total de preguntas y progreso del estudiante logueado.
POST	<code>/cursos</code>	admin	Crea curso e inicializa su carpeta en <code>writable/cursos/{id}</code> .
PUT	<code>/cursos/{id}</code>	admin	Actualiza datos del curso.
POST	<code>/cursos/{id}/portada</code>	admin	Sube imagen de portada (<code>public/uploads/cursos/{id}/</code> , máx 5 MB).
GET	<code>/cursos/{id}/secciones</code>	Público	Lista secciones de un curso ordenadas.
POST	<code>/cursos/{id}/secciones</code>	admin	Crea sección subiendo un PDF (multipart).
PUT / POST	<code>/cursos/{id}/secciones/{idSec}</code>	admin	Actualiza sección ± reemplazo de PDF.
PUT	<code>/cursos/{id}/secciones/{idSec}/eliminar</code>	admin	Inactiva sección (Estado = 0). No borra el archivo físico.

Método	Ruta	Auth	Descripción
GET	/cursos/{id}/secciones/descargar	Público	Descarga el PDF de una sección (parámetro archivoUrl).
GET	/cursos/{id}/preguntas-sugeridas	admin	Genera preguntas con OpenAI a partir de los PDFs.
POST	/cursos/{id}/inicio	auth	Registra FechaInicio. Reinicia progreso si ya existía.
POST	/cursos/{id}/finalizacion	auth	Guarda UltimaCalificacion (0–100), Aprobo y VariablesSeguimiento.

7.3 Preguntas, evidencias, noticias y contacto

Método	Ruta	Auth	Descripción
GET	/preguntas	Público	Lista preguntas (filtro opcional por idCurso, incluye opciones).
POST	/preguntas	admin	Crea pregunta con sus opciones.
PUT	/preguntas/update/{id}	admin	Actualiza pregunta.
DELETE	/preguntas/{id}	admin	Elimina pregunta.
GET	/evidencias	Público	Lista todas las evidencias.
GET	/evidencias/activas	Público	Lista sólo las que tienen Estado = 1.
GET	/evidencias/{id}	Público	Detalle de evidencia.
POST	/evidencias	admin	Crea evidencia.
PUT	/evidencias/{id}	admin	Actualiza evidencia.
GET	/noticias-eventos	Público	Lista de noticias/eventos.
GET	/noticias-eventos/{id}	Público	Detalle.
POST	/noticias-eventos	admin	Crea noticia/evento.
PUT	/noticias-eventos/{id}	admin	Actualiza.
GET	/contacto	Público	Lista mensajes de contacto.
POST	/contacto	Público	Registra mensaje del formulario público.
OPTIONS	(:any)	Público	Preflight CORS (responde 200).

Ejemplos de uso

Login

```
curl -X POST http://localhost:8080/login \
-H 'Content-Type: application/json' \
-d '{"Correo":"demo@etitec.edu.co","Contraseña":"secreta123"}'
```

Respuesta:

```
{
  "success": true,
  "message": "Login exitoso",
  "token": "eyJhbGciOiJIUzI1NiIs..."
}
```

Listar estudiantes (admin)

```
curl http://localhost:8080/usuario/estudiantes \
-H 'Authorization: Bearer eyJhbGciOiJIUzI1NiIs...'
```

Subir PDF de una sección

```
curl -X POST http://localhost:8080/cursos/11/secciones \
-H 'Authorization: Bearer <TOKEN_ADMIN>' \
-F 'Nombre=Introducción a los biopolímeros' \
-F 'Orden=1' \
-F 'archivo=@/ruta/local/seccion_1.pdf'
```

8. Servicios de negocio

La capa **Services** concentra la lógica que cruza varios modelos o interactúa con el filesystem y APIs externas. Los controllers invocan los services a través de `\Config\Services::...()` (singletons).

Service	Responsabilidad
CursoService	Crea cursos y su carpeta en <i>writable/cursos/{id}</i> . Valida unicidad de nombre (case-insensitive).
SeccionCursoService	Valida PDFs, mueve archivos a la carpeta del curso, actualiza o inactiva secciones, resuelve la ruta absoluta para descarga controlada.
ImageUploadService	Sube imágenes públicas (<i>public/uploads/<subdir>/</i>) con validación MIME y tamaño. Elimina imágenes previas evitando path-traversal. Límites: 2 MB (perfil) / 5 MB (portada).
CursoDesarrolloService	Registra el avance del estudiante: crea o reinicia un registro en <i>CursoDesarrolloEstudiante</i> al inicio, y lo actualiza al finalizar. Valida <i>UltimaCalificacion</i> (0–100), <i>Aprobo</i> (0/1) y <i>VariablesSeguimiento</i> (JSON).
PreguntasSugeridasService	Invoca OpenAI (Files + Responses API) por cada sección del curso para generar preguntas en formato JSON normalizado. Reintenta ante rate-limit. Límite: 20 MB por PDF.

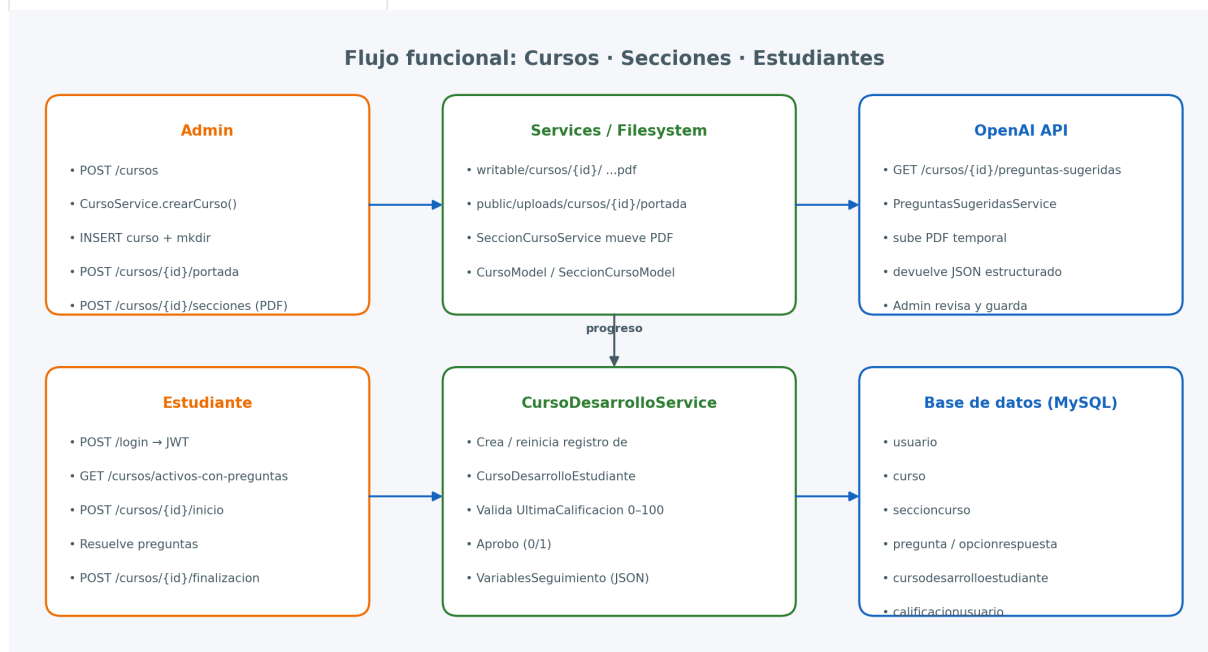


Fig. 4 · Flujo funcional de cursos (admin + estudiante).

9. Integración con OpenAI

El endpoint `GET /cursos/{id}/preguntas-sugeridas` permite a un admin generar automáticamente preguntas a partir de los PDFs de las secciones activas del curso. El flujo real vive en `app/Services/PreguntasSugeridasService.php`.

Pasos

- Recorre las secciones activas del curso ordenadas por *Orden*.
- Para cada PDF: valida MIME, verifica tamaño ≤ 20 MB, sube el archivo con la *Files API* de OpenAI.
- Llama a la *Responses API* con un prompt que pide **mínimo 10 preguntas** por PDF y tipos válidos: *multiple, única, falso_verdadero, autocompletar*.
- Parsea la respuesta JSON (con parsing tolerante a texto antes/después del bloque) y normaliza la estructura.
- Devuelve al admin un payload con las secciones y sus preguntas; **no** se insertan en la BD automáticamente — el admin las revisa y las crea con **POST /preguntas**.

Límites operativos

- Timeout HTTP: 120 s; reintentos con back-off (5 s y 15 s) ante *rate_limit*.
- Coste: cada llamada consume tokens de OpenAI; monitorea uso en la cuenta.
- Si *OPENAI_API_KEY* no está definida, el endpoint devuelve 500. Configúrala en el *.env*.

10. Almacenamiento de archivos

Ubicación	Contenido	Visibilidad
public/uploads/usuarios/	Fotos de perfil de usuarios	Público (URL directa)
public/uploads/cursos/{idCurso}/	Imagen de portada del curso	Público (URL directa)
writable/cursos/{idCurso}/	PDFs originales de cada sección del curso	Privado. Se descargan via endpoint /descargar que valida pertenencia.
writable/cache/	Cache interno de CI4	Privado
writable/logs/	Log application / php	Privado
writable/session/	Sesiones (si se activan)	Privado
writable/uploads/	Archivos temporales	Privado

Validaciones de upload

- Imágenes: JPEG, PNG, WebP, GIF. Detección MIME con *mime_content_type* + *finfo* + *getimagesize* como fallback.
- PDFs: MIME debe ser *application/pdf*. Se renombran con timestamp + hash para evitar colisiones.
- Path traversal: **ImageUploadService** y **SeccionCursoService** resuelven rutas absolutas y verifican que caigan dentro de las carpetas esperadas.

11. Convenciones, pendientes y recomendaciones

Convenciones adoptadas

- Respuestas: siempre JSON con *success* + *message*; en errores de validación se devuelve **200** con *success: false* (no 4xx). Si construyes un cliente, no dependas del status code para distinguir validaciones.
- Columnas *FechaCreacion* / *FechaActualizacion* se setean manualmente en cada insert/update.
- No se usan *Entities* ni *Form Validation* de CI4; la validación es manual dentro del controller.
- Swagger/OpenAPI se regenera con `vendor/bin/openapi app --output swagger.json` (revisa `app/Controllers/OpenApi/` para la metadata global).

Deudas técnicas conocidas

- **Código duplicado de CORS:** algunos controllers setean headers manualmente además del filtro *Cors*. Unificar en el filtro.
- **idRol hard-codeado** (1 estudiante, 2 admin). Considerar constantes o enum.
- **Respuesta de errores:** mezcla 200/400/401/403/404. Unificar una estrategia (RFC 7807 o similar).
- **Rate limiting** no está implementado; expone al endpoint de OpenAI y al envío de correo a abuso.
- **Tests:** *tests/* trae el esqueleto pero no hay pruebas específicas del dominio.
- **Migraciones** incompletas respecto al DDL original (falta *idCurso* en *Pregunta*, *FotoPerfil* en *Usuario*, etc.). Alinear antes de desplegar nuevos ambientes.

Próximos pasos sugeridos

- Separar configuración por ambiente (*.env.development* / *.env.production*) y automatizar despliegue con GitHub Actions.
- Añadir pruebas *Feature* (Codelgniter 4) para los endpoints críticos de auth y cursos.
- Implementar paginación en los endpoints que retornan listas grandes (*cursos*, *preguntas*, *noticias-eventos*).
- Reducir duración del JWT (15–30 min) y agregar refresh tokens.
- Registrar auditoría (quién creó / actualizó qué) en tablas de contenido.
- Mover la generación de preguntas OpenAI a un job en background; hoy bloquea hasta 2 min al admin.

12. Contacto

Manuel Alberto Rojas Martínez — Investigador y Desarrollador. Correo: manuelrojas13@gmail.com · GitHub: github.com/manuelrojas13.

Jaime Alberto Paez — Docente e Investigador Supervisor (ETITC).

Proyecto realizado para la **Escuela Tecnológica Instituto Técnico Central (ETITC)**. Todos los derechos reservados.